



وزارة التربية الوطنية
والتعليم الأولي والرياضة

+٠٢٠٧٠٠+ | %٠X٢٤ ٠٠٢%O
^ %٠٠H٢^ ٠٢J٧٠O% ^ +%H٢+

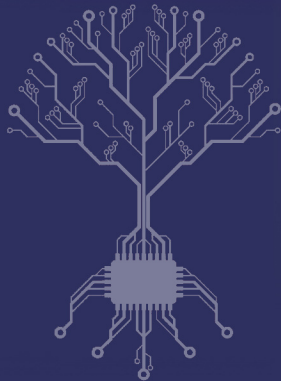
BASE DE DONNÉES (1/3)

1 Introduction

2 SQL

- Langage de Définition de Données (*LDD*)
- Langage de Manipulation de Données (*LMD*)

INTRODUCTION



Après avoir établi le *MCD* et le *MLD*, il faut adapter ce dernier à un *SGBD* pour créer le *MPD* en utilisant un langage appelé *SQL* (Structured Query Language).

Système de Gestion de Base de Données

Un *SGBD* est un logiciel (e.g. *MySQL*, *Oracle*, *SQLite*, ...) qui permet de gérer une *BD* pour définir et manipuler les données, tout en garantissant leur cohérence et intégrité.

Le *SGBD* utilisé dans la suite est *SQLite*.

Selon le modèle relationnel, la base de données est composée d'un ensemble de relations (tables 2D) dans lesquelles sont placées les données, ainsi que les liens entre ces différentes relations.

Exemples :

idstudent	firstname	lastname	field	average	idschool
24142356	ahmad	bachir	economics	15	CP1733
24167849	omar	mohammadi	economics	14	CP1733
241677834	fatima	najma	mathematics	18	CP1710

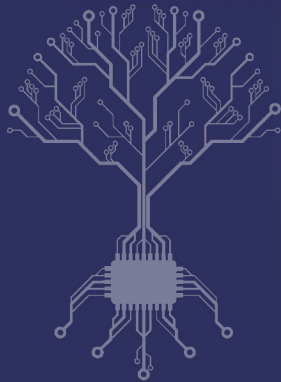
Table – Student

idschool	name	city
CP1710	Charif Alidrissi	Taza
CP1733	Omar Alkhayam	Rabat

Table – School

Les tables ne représentent qu'une abstraction de l'enregistrement physique des données en mémoire.

SQL



SQL est un langage informatique normalisé servant à exploiter des bases de données relationnelles. Il comporte plusieurs sous langages comme :

- Langage de Définition de Données (*LDD*)
- Langage de Manipulation de Données (*LMD*)
 - ▶ Langage d'Interrogation de Données (*LID*)

LDD

Le *LDD* permet de créer, de modifier et de supprimer une table.

CRÉATION

La création d'une table consiste à définir :

- Son nom ;
- Ses colonnes (leurs noms, types et contraintes d'intégrité) ;
- Ses contraintes d'intégrité.

CRÉATION

Syntaxe

```
CREATE TABLE [IF NOT EXISTS] nomTable (  
  nomColonne1 type contraintesColonne1,  
  nomColonne2 type contraintesColonne2,  
  ...  
  contraintesTable,  
  ...  
);
```

CRÉATION : TYPES DE DONNÉES

- TEXT : Chaîne de caractères
- INTEGER : Nombre entier
- REAL : Nombre réel

CRÉATION : CONTRAINTES D'INTÉGRITÉ

Définition

Une contrainte d'intégrité est une propriété que les données d'une base doivent vérifier pour garantir leur cohérence.

CRÉATION : CONTRAINTES DE COLONNE

- **UNIQUE** : Toutes les valeurs de la colonne doivent être différentes.
- **NOT NULL** ou **NULL** : Les valeurs de la colonne doivent être spécifiées ou non.
- **PRIMARY KEY** : Désigne la colonne comme clé primaire de la table.
- **DEFAULT** valeur : Permet de spécifier la valeur par défaut de la colonne.

CRÉATION : CONTRAINTES DE COLONNE

Exemple

La table *school*(*name*, *address*), où :

- *name* est une chaîne de caractères qui constitue la clé primaire,
 - *address* est une chaîne de caractères obligatoire,
- peut être créée comme suit :

```
CREATE TABLE school (  
  name TEXT PRIMARY KEY,  
  address TEXT NOT NULL  
);
```

CRÉATION : CONTRAINTES DE COLONNE

- **CHECK (condition)** : Vérifie lors de l'insertion des enregistrements (lignes) que l'attribut vérifie la condition.

Les opérateurs de comparaison :

- ▶ `=, !=, <, >, <=, >=`
- ▶ `BETWEEN value1 AND value2`
- ▶ `IN` ou `NOT IN (value1, value2, ...)`
- ▶ `LIKE` : permet d'utiliser des jokers (`%` pour une chaîne de caractères de longueur quelconque et `_` pour un seul caractère)

Exemple

```
CREATE TABLE student (  
  idstudent INTEGER PRIMARY KEY,  
  firstname TEXT NOT NULL,  
  lastname TEXT NOT NULL,  
  field TEXT NOT NULL,  
  average REAL NOT NULL CHECK (average BETWEEN 0 AND 20)  
);
```

CRÉATION : CONTRAINTES DE TABLE

Les contraintes de tables portent sur plusieurs attributs de la table sur laquelle elles sont définies :

- PRIMARY KEY (colonne1, colonne2, ...) : Désigne la concaténation des attributs cités comme clé primaire de la table.
- FOREIGN KEY (colonne) REFERENCES table (colonne) : Identifie une ou plusieurs colonnes comme étant clé étrangère.

CRÉATION : CONTRAINTES DE TABLE

Exemple

Soit les schémas :

- professor(idprof, firstname, lastname, mail)
- course(idcourse, name, field)
- session(#idprof, #idcourse, date)

CRÉATION : CONTRAINTES DE TABLE

Exemple

```
CREATE TABLE professor(  
  idprof INTEGER PRIMARY KEY,  
  firstname TEXT NOT NULL,  
  lastname TEXT NOT NULL,  
  mail TEXT NOT NULL CHECK (mail LIKE '%@%.%')  
);
```

CRÉATION : CONTRAINTES DE TABLE

Exemple

```
CREATE TABLE course (  
  idcourse INTEGER PRIMARY KEY,  
  name TEXT NOT NULL,  
  field TEXT NOT NULL  
);
```

CRÉATION : CONTRAINTES DE TABLE

Exemple

```
CREATE TABLE session(  
  idprof INTEGER,  
  idcourse INTEGER,  
  date TEXT,  
  PRIMARY KEY (idprof, idcourse),  
  FOREIGN KEY (idprof) REFERENCES professor (idprof),  
  FOREIGN KEY (idcourse) REFERENCES course (idcourse)  
);
```

MODIFICATION

■ Ajout d'une colonne

- ▶ Syntaxe : `ALTER TABLE nomTable ADD (nomColonne type contraintes);`
- ▶ Exemple : `ALTER TABLE student ADD (age INTEGER);`

■ Renommage d'une colonne

- ▶ Syntaxe : `ALTER TABLE nomTable RENAME ancienNom TO nouveauNom;`

■ Suppression d'une colonne

- ▶ Syntaxe : `ALTER TABLE nomTable DROP nomColonne;`
- ▶ Exemple : `ALTER TABLE student DROP age;`

SUPPRESSION

```
DROP TABLE nomTable;
```

MODIFICATION ET SUPPRESSION

La modification ou suppression d'une ligne dans une table peut être impossible s'il existe des lignes de clés étrangères dans d'autres tables référençant la clé primaire de cette ligne. Pour éviter ce genre de problème, on peut ajouter à la contrainte FOREIGN KEY des options :

MODIFICATION ET SUPPRESSION

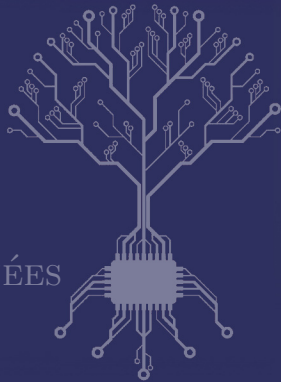
■ En cas de modification :

- ▶ ON UPDATE CASCADE : modifier aussi les valeurs des clés étrangères.
- ▶ ON UPDATE SET NULL : mettre NULL à la place des valeurs des clés étrangères.

■ En cas de suppression :

- ▶ ON DELETE CASCADE : supprimer les lignes concernées.
- ▶ ON DELETE SET NULL : mettre NULL à la place des valeurs des clés étrangères.

SQL : LANGAGE DE MANIPULATION DE DONNÉES



Le *LMD* permet d'insérer, de modifier et de supprimer les lignes d'une table.

INSERTION DE LIGNES

La commande INSERT permet d'insérer une ligne dans une table en spécifiant les valeurs à insérer.

Syntaxe

```
INSERT INTO nomTable VALUES  
(valeur11, valeur12, . . . ),  
(valeur21, valeur22, . . . ),  
... ;
```

INSERTION DE LIGNES

Exemple

```
INSERT INTO student VALUES  
(251423561, 'ahmad', 'bachir', 15, 'economics'),  
(251677830, 'fatima', 'mohammadi', 19, 'mathematics');
```

MODIFICATION DE LIGNES

La commande UPDATE permet de modifier les valeurs d'une ou plusieurs colonnes, dans une ou plusieurs lignes, selon une condition.

Syntaxe

```
UPDATE nomTable SET nomColonne = expression, ... WHERE condition ;
```

En l'absence d'une clause WHERE, toutes les lignes sont mises à jour.

MODIFICATION DE LIGNES

Exemple

Soit la table *employee*(*idemployee*, *name*, *salary*, *departement*) ; Pour augmenter de 10% le salaire des employés du département commercial on exécute la requête :

```
UPDATE employee SET salary = salary * 1.1 WHERE departement = 'commerce' ;
```

SUPPRESSION DE LIGNES

Syntaxe

```
DELETE FROM nomTable WHERE condition ;
```

Exemple

La requête :

```
DELETE FROM employee WHERE departement = 'commerce' ;
```

supprime tous les employés du département commercial.